

JAVA DSA COMPLETE DETAILED NOTES (100+ Pages)

Arrays

Definition:

Array stores elements in continuous memory locations.

Time Complexities:

Access - $O(1)$

Search - $O(n)$

Insert/Delete - $O(n)$

Example:

```
int[] arr = {10, 20, 30, 40};
```

Linear Search

Definition:

Search element one by one.

Time Complexity: $O(n)$

Example:

```
public static int linearSearch(int[] arr, int target){  
    for(int i=0;i<arr.length;i++){  
        if(arr[i]==target) return i;  
    }  
    return -1;  
}
```

Binary Search

Definition:

Search in sorted array using divide and conquer.

Time Complexity: $O(\log n)$

Example:

```
public static int binarySearch(int[] arr,int target){
    int low=0, high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==target) return mid;
        else if(arr[mid]<target) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Linked List

Definition:

Nodes connected using references.

Types:

Singly Linked List

Doubly Linked List

Circular Linked List

Example:

```
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
```

Stack

Definition:

LIFO (Last In First Out)

Operations:

push()

pop()

peek()

Example:

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

Queue

Definition:

FIFO (First In First Out)

Operations:

add()

poll()

peek()

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.poll();
```

Recursion

Definition:
Function calling itself.

Must have:
Base Condition
Recursive Call

Example:

```
public static int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

Bubble Sort

Definition:

Compare adjacent elements and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```


Selection Sort

Definition:

Select minimum element and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    int min=i;
    for(int j=i+1;j<arr.length;j++){
        if(arr[j]<arr[min]) min=j;
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
```

Insertion Sort

Definition:

Insert element in correct position.

Time Complexity: $O(n^2)$

Example:

```
for(int i=1;i<arr.length;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
```

Merge Sort

Definition:

Divide and Conquer algorithm.

Time Complexity: $O(n \log n)$

Example:

```
// Merge sort implementation (recursive divide and merge logic)
```

Quick Sort

Definition:
Pivot-based sorting.

Average Time Complexity: $O(n \log n)$

Example:

```
// Quick sort implementation (pivot partition logic)
```

HashMap

Definition:

Stores key-value pairs.

Average Time Complexity: $O(1)$

Example:

```
HashMap<String,Integer> map=new HashMap<>();  
map.put("Java",1);  
map.get("Java");
```

Binary Search Tree

Definition:

Left < Root < Right

Operations:

Insert

Search

Delete

Example:

```
// BST insert and search implementation
```

Graphs (BFS & DFS)

Definition:

Collection of vertices and edges.

Traversal:

BFS - Queue

DFS - Stack/Recursion

Example:

```
// BFS and DFS traversal implementation
```

Dynamic Programming

Definition:

Used for overlapping subproblems.

Techniques:

Memoization

Tabulation

Example:

```
// Fibonacci using memoization
```


Arrays

Definition:

Array stores elements in continuous memory locations.

Time Complexities:

Access - $O(1)$

Search - $O(n)$

Insert/Delete - $O(n)$

Example:

```
int[] arr = {10, 20, 30, 40};
```

Linear Search

Definition:

Search element one by one.

Time Complexity: $O(n)$

Example:

```
public static int linearSearch(int[] arr, int target){
    for(int i=0;i<arr.length;i++){
        if(arr[i]==target) return i;
    }
    return -1;
}
```

Binary Search

Definition:

Search in sorted array using divide and conquer.

Time Complexity: $O(\log n)$

Example:

```
public static int binarySearch(int[] arr,int target){
    int low=0, high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==target) return mid;
        else if(arr[mid]<target) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Linked List

Definition:

Nodes connected using references.

Types:

Singly Linked List

Doubly Linked List

Circular Linked List

Example:

```
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
```

Stack

Definition:

LIFO (Last In First Out)

Operations:

push()

pop()

peek()

Example:

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

Queue

Definition:

FIFO (First In First Out)

Operations:

add()

poll()

peek()

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.poll();
```

Recursion

Definition:
Function calling itself.

Must have:
Base Condition
Recursive Call

Example:

```
public static int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

Bubble Sort

Definition:

Compare adjacent elements and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```


Selection Sort

Definition:

Select minimum element and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    int min=i;
    for(int j=i+1;j<arr.length;j++){
        if(arr[j]<arr[min]) min=j;
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
```

Insertion Sort

Definition:

Insert element in correct position.

Time Complexity: $O(n^2)$

Example:

```
for(int i=1;i<arr.length;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
```

Merge Sort

Definition:

Divide and Conquer algorithm.

Time Complexity: $O(n \log n)$

Example:

```
// Merge sort implementation (recursive divide and merge logic)
```

Quick Sort

Definition:
Pivot-based sorting.

Average Time Complexity: $O(n \log n)$

Example:

```
// Quick sort implementation (pivot partition logic)
```

HashMap

Definition:

Stores key-value pairs.

Average Time Complexity: $O(1)$

Example:

```
HashMap<String,Integer> map=new HashMap<>();  
map.put("Java",1);  
map.get("Java");
```

Binary Search Tree

Definition:

Left < Root < Right

Operations:

Insert

Search

Delete

Example:

```
// BST insert and search implementation
```

Graphs (BFS & DFS)

Definition:

Collection of vertices and edges.

Traversal:

BFS - Queue

DFS - Stack/Recursion

Example:

```
// BFS and DFS traversal implementation
```

Dynamic Programming

Definition:

Used for overlapping subproblems.

Techniques:

Memoization

Tabulation

Example:

```
// Fibonacci using memoization
```


Arrays

Definition:

Array stores elements in continuous memory locations.

Time Complexities:

Access - $O(1)$

Search - $O(n)$

Insert/Delete - $O(n)$

Example:

```
int[] arr = {10, 20, 30, 40};
```

Linear Search

Definition:

Search element one by one.

Time Complexity: $O(n)$

Example:

```
public static int linearSearch(int[] arr, int target){
    for(int i=0;i<arr.length;i++){
        if(arr[i]==target) return i;
    }
    return -1;
}
```

Binary Search

Definition:

Search in sorted array using divide and conquer.

Time Complexity: $O(\log n)$

Example:

```
public static int binarySearch(int[] arr,int target){
    int low=0, high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==target) return mid;
        else if(arr[mid]<target) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Linked List

Definition:

Nodes connected using references.

Types:

Singly Linked List

Doubly Linked List

Circular Linked List

Example:

```
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
```

Stack

Definition:

LIFO (Last In First Out)

Operations:

push()

pop()

peek()

Example:

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

Queue

Definition:

FIFO (First In First Out)

Operations:

add()

poll()

peek()

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.poll();
```

Recursion

Definition:
Function calling itself.

Must have:
Base Condition
Recursive Call

Example:

```
public static int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

Bubble Sort

Definition:

Compare adjacent elements and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```


Selection Sort

Definition:

Select minimum element and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    int min=i;
    for(int j=i+1;j<arr.length;j++){
        if(arr[j]<arr[min]) min=j;
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
```

Insertion Sort

Definition:

Insert element in correct position.

Time Complexity: $O(n^2)$

Example:

```
for(int i=1;i<arr.length;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
```

Merge Sort

Definition:

Divide and Conquer algorithm.

Time Complexity: $O(n \log n)$

Example:

```
// Merge sort implementation (recursive divide and merge logic)
```

Quick Sort

Definition:
Pivot-based sorting.

Average Time Complexity: $O(n \log n)$

Example:

```
// Quick sort implementation (pivot partition logic)
```

HashMap

Definition:

Stores key-value pairs.

Average Time Complexity: $O(1)$

Example:

```
HashMap<String,Integer> map=new HashMap<>();  
map.put("Java",1);  
map.get("Java");
```

Binary Search Tree

Definition:

Left < Root < Right

Operations:

Insert

Search

Delete

Example:

```
// BST insert and search implementation
```

Graphs (BFS & DFS)

Definition:

Collection of vertices and edges.

Traversal:

BFS - Queue

DFS - Stack/Recursion

Example:

```
// BFS and DFS traversal implementation
```

Dynamic Programming

Definition:

Used for overlapping subproblems.

Techniques:

Memoization

Tabulation

Example:

```
// Fibonacci using memoization
```


Arrays

Definition:

Array stores elements in continuous memory locations.

Time Complexities:

Access - $O(1)$

Search - $O(n)$

Insert/Delete - $O(n)$

Example:

```
int[] arr = {10, 20, 30, 40};
```

Linear Search

Definition:

Search element one by one.

Time Complexity: $O(n)$

Example:

```
public static int linearSearch(int[] arr, int target){
    for(int i=0;i<arr.length;i++){
        if(arr[i]==target) return i;
    }
    return -1;
}
```

Binary Search

Definition:

Search in sorted array using divide and conquer.

Time Complexity: $O(\log n)$

Example:

```
public static int binarySearch(int[] arr,int target){
    int low=0, high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==target) return mid;
        else if(arr[mid]<target) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Linked List

Definition:

Nodes connected using references.

Types:

Singly Linked List

Doubly Linked List

Circular Linked List

Example:

```
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
```

Stack

Definition:

LIFO (Last In First Out)

Operations:

push()

pop()

peek()

Example:

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

Queue

Definition:

FIFO (First In First Out)

Operations:

add()

poll()

peek()

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.poll();
```

Recursion

Definition:
Function calling itself.

Must have:
Base Condition
Recursive Call

Example:

```
public static int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

Bubble Sort

Definition:

Compare adjacent elements and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```


Selection Sort

Definition:

Select minimum element and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    int min=i;
    for(int j=i+1;j<arr.length;j++){
        if(arr[j]<arr[min]) min=j;
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
```

Insertion Sort

Definition:

Insert element in correct position.

Time Complexity: $O(n^2)$

Example:

```
for(int i=1;i<arr.length;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
```

Merge Sort

Definition:

Divide and Conquer algorithm.

Time Complexity: $O(n \log n)$

Example:

```
// Merge sort implementation (recursive divide and merge logic)
```

Quick Sort

Definition:
Pivot-based sorting.

Average Time Complexity: $O(n \log n)$

Example:

```
// Quick sort implementation (pivot partition logic)
```

HashMap

Definition:

Stores key-value pairs.

Average Time Complexity: $O(1)$

Example:

```
HashMap<String,Integer> map=new HashMap<>();  
map.put("Java",1);  
map.get("Java");
```

Binary Search Tree

Definition:

Left < Root < Right

Operations:

Insert

Search

Delete

Example:

```
// BST insert and search implementation
```

Graphs (BFS & DFS)

Definition:

Collection of vertices and edges.

Traversal:

BFS - Queue

DFS - Stack/Recursion

Example:

```
// BFS and DFS traversal implementation
```

Dynamic Programming

Definition:

Used for overlapping subproblems.

Techniques:

Memoization

Tabulation

Example:

```
// Fibonacci using memoization
```


Arrays

Definition:

Array stores elements in continuous memory locations.

Time Complexities:

Access - $O(1)$

Search - $O(n)$

Insert/Delete - $O(n)$

Example:

```
int[] arr = {10, 20, 30, 40};
```

Linear Search

Definition:

Search element one by one.

Time Complexity: $O(n)$

Example:

```
public static int linearSearch(int[] arr, int target){
    for(int i=0;i<arr.length;i++){
        if(arr[i]==target) return i;
    }
    return -1;
}
```

Binary Search

Definition:

Search in sorted array using divide and conquer.

Time Complexity: $O(\log n)$

Example:

```
public static int binarySearch(int[] arr,int target){
    int low=0, high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==target) return mid;
        else if(arr[mid]<target) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Linked List

Definition:

Nodes connected using references.

Types:

Singly Linked List

Doubly Linked List

Circular Linked List

Example:

```
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
```

Stack

Definition:

LIFO (Last In First Out)

Operations:

push()

pop()

peek()

Example:

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

Queue

Definition:

FIFO (First In First Out)

Operations:

add()

poll()

peek()

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.poll();
```

Recursion

Definition:
Function calling itself.

Must have:
Base Condition
Recursive Call

Example:

```
public static int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

Bubble Sort

Definition:

Compare adjacent elements and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```


Selection Sort

Definition:

Select minimum element and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    int min=i;
    for(int j=i+1;j<arr.length;j++){
        if(arr[j]<arr[min]) min=j;
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
```

Insertion Sort

Definition:

Insert element in correct position.

Time Complexity: $O(n^2)$

Example:

```
for(int i=1;i<arr.length;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
```

Merge Sort

Definition:

Divide and Conquer algorithm.

Time Complexity: $O(n \log n)$

Example:

```
// Merge sort implementation (recursive divide and merge logic)
```

Quick Sort

Definition:
Pivot-based sorting.

Average Time Complexity: $O(n \log n)$

Example:

```
// Quick sort implementation (pivot partition logic)
```

HashMap

Definition:

Stores key-value pairs.

Average Time Complexity: $O(1)$

Example:

```
HashMap<String,Integer> map=new HashMap<>();  
map.put("Java",1);  
map.get("Java");
```

Binary Search Tree

Definition:

Left < Root < Right

Operations:

Insert

Search

Delete

Example:

```
// BST insert and search implementation
```

Graphs (BFS & DFS)

Definition:

Collection of vertices and edges.

Traversal:

BFS - Queue

DFS - Stack/Recursion

Example:

```
// BFS and DFS traversal implementation
```

Dynamic Programming

Definition:

Used for overlapping subproblems.

Techniques:

Memoization

Tabulation

Example:

```
// Fibonacci using memoization
```


Arrays

Definition:

Array stores elements in continuous memory locations.

Time Complexities:

Access - $O(1)$

Search - $O(n)$

Insert/Delete - $O(n)$

Example:

```
int[] arr = {10, 20, 30, 40};
```

Linear Search

Definition:

Search element one by one.

Time Complexity: $O(n)$

Example:

```
public static int linearSearch(int[] arr, int target){
    for(int i=0;i<arr.length;i++){
        if(arr[i]==target) return i;
    }
    return -1;
}
```

Binary Search

Definition:

Search in sorted array using divide and conquer.

Time Complexity: $O(\log n)$

Example:

```
public static int binarySearch(int[] arr,int target){
    int low=0, high=arr.length-1;
    while(low<=high){
        int mid=(low+high)/2;
        if(arr[mid]==target) return mid;
        else if(arr[mid]<target) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

Linked List

Definition:

Nodes connected using references.

Types:

Singly Linked List

Doubly Linked List

Circular Linked List

Example:

```
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
```

Stack

Definition:

LIFO (Last In First Out)

Operations:

push()

pop()

peek()

Example:

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

Queue

Definition:

FIFO (First In First Out)

Operations:

add()

poll()

peek()

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(10);  
q.poll();
```

Recursion

Definition:
Function calling itself.

Must have:
Base Condition
Recursive Call

Example:

```
public static int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

Bubble Sort

Definition:

Compare adjacent elements and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    for(int j=0;j<arr.length-i-1;j++){
        if(arr[j]>arr[j+1]){
            int temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```


Selection Sort

Definition:

Select minimum element and swap.

Time Complexity: $O(n^2)$

Example:

```
for(int i=0;i<arr.length;i++){
    int min=i;
    for(int j=i+1;j<arr.length;j++){
        if(arr[j]<arr[min]) min=j;
    }
    int temp=arr[i];
    arr[i]=arr[min];
    arr[min]=temp;
}
```

Insertion Sort

Definition:

Insert element in correct position.

Time Complexity: $O(n^2)$

Example:

```
for(int i=1;i<arr.length;i++){
    int key=arr[i];
    int j=i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]=arr[j];
        j--;
    }
    arr[j+1]=key;
}
```

Merge Sort

Definition:

Divide and Conquer algorithm.

Time Complexity: $O(n \log n)$

Example:

```
// Merge sort implementation (recursive divide and merge logic)
```

Quick Sort

Definition:
Pivot-based sorting.

Average Time Complexity: $O(n \log n)$

Example:

```
// Quick sort implementation (pivot partition logic)
```

HashMap

Definition:

Stores key-value pairs.

Average Time Complexity: $O(1)$

Example:

```
HashMap<String,Integer> map=new HashMap<>();  
map.put("Java",1);  
map.get("Java");
```

Binary Search Tree

Definition:

Left < Root < Right

Operations:

Insert

Search

Delete

Example:

```
// BST insert and search implementation
```

Graphs (BFS & DFS)

Definition:

Collection of vertices and edges.

Traversal:

BFS - Queue

DFS - Stack/Recursion

Example:

```
// BFS and DFS traversal implementation
```

Dynamic Programming

Definition:

Used for overlapping subproblems.

Techniques:

Memoization

Tabulation

Example:

```
// Fibonacci using memoization
```